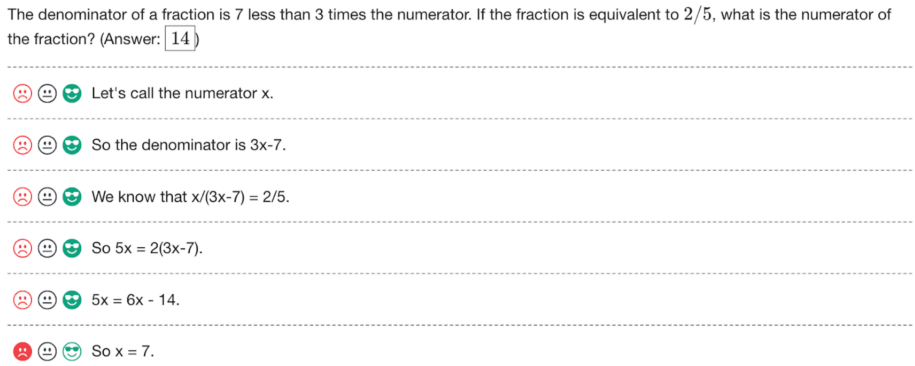

Figure 1: A screenshot of the interface used to collect feedback for each step in a solution.

math-relevant tokens, which we call MathMix. Similar to Lewkowycz et al. (2022), we find that this improves the model's mathematical reasoning capabilities. Details on how this dataset was constructed can be found in Appendix A.

2.3 Generator

To make parsing individual steps easier, we train the generator to produce solutions in a newline delimited step-by-step format. Specifically, we few-shot generate solutions to MATH training problems, filter to those that reach the correct final answer, and finetune the base model on this dataset for a single epoch. This step is not intended to teach the generator new skills; it is intended only to teach the generator to produce solutions in the desired format.

2.4 Data Collection

To collect process supervision data, we present human data-labelers with step-by-step solutions to MATH problems sampled by the large-scale generator. Their task is to assign each step in the solution a label of *positive*, *negative*, or *neutral*, as shown in Figure 1. A positive label indicates that the step is correct and reasonable. A negative label indicates that the step is either incorrect or unreasonable. A neutral label indicates ambiguity. In practice, a step may be labelled neutral if it is subtly misleading, or if it is a poor suggestion that is technically still valid. We permit neutral labels since this allows us to defer the decision about how to handle ambiguity: at test time, we can treat neutral labels as either positive or negative. A more detailed description of the labelling instructions is provided in Appendix D.

We label solutions exclusively from the large-scale generator in order to maximize the value of our limited human-data resource. We refer to the entire dataset of step-level labels collected as PRM800K. The PRM800K training set contains 800K step-level labels across 75K solutions to 12K problems. To

minimize overfitting, we include data from 4.5K MATH test problems in the PRM800K training set, and we therefore evaluate our models only on the remaining 500 MATH test problems. More details about this test set can be found in Appendix C.

During data collection, we must decide which solutions to surface to data-labelers. The most straightforward strategy is to uniformly surface solutions produced by the generator. However, if we surface solutions that make obvious errors, the human feedback we get is less valuable. We would prefer to surface solutions that are more likely to fool our best reward model. To that end, we attempt to strategically select which solutions to show data-labelers. Specifically, we choose to surface *convincing wrong-answer* solutions. We use the term *convincing* to refer to solutions that are rated highly by our current best PRM, and we use *wrong-answer* to refer to solutions that reach an incorrect final answer. We use this slightly verbose phrasing to emphasize the fact that correctness is determined solely by checking the final answer, a process which occasionally leads to misgraded solutions. We expect to gain more information from labeling convincing wrong-answer solutions, since we know the PRM is mistaken about at least one step in each such solution.

In addition to using this selection strategy, we also iteratively re-train our PRM using the latest data at several points in the data collection process. At each iteration, we generate N solutions per problem and surface only the top K most convincing wrong-answer solutions to data-labelers. We experiment with either applying this top-K filtering at a problem level (K solutions per problem) or globally across the dataset (K solutions in total, unequally distributed among problems). Since the data collection process is expensive, it was not feasible to conduct at-scale ablations of these decisions. However, we perform several surrogate ablations in Section 4, using our largest PRM as a labelling oracle for a smaller PRM. More details about data collection can be found in Appendix B.

2.5 Outcome-supervised Reward Models (ORMs)

We train ORMs following a similar methodology to Cobbe et al. (2021). We uniformly sample a fixed number of solutions per problem from the generator, and we train the ORM to predict whether each solution is correct or incorrect. In practice, we usually determine correctness by automatically checking the final answer, but in principle these labels could be provided by humans. At test time, we use the ORM's prediction at the final token as the overall score for the solution. We note the automatic grading used to determine ORM targets is not perfectly reliable: *false positives* solutions that reach the correct answer with incorrect reasoning will be misgraded. We discuss additional ORM training details in Appendix E.

2.6 Process-supervised Reward Models (PRMs)

We train PRMs to predict the correctness of each step after the last token in each step. This prediction takes the form of a single token, and we maximize the

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x)\cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

I notice that the given polynomial has even degree and only even powers of x, so I can try to make a substitution to simplify it.

Let $y = x^4$, then the polynomial becomes $y^2 + 3y - 4$, which is a quadratic equation.

I can factor this quadratic equation as $(y + 4)(y - 1)$, so the original polynomial is $(x^4 + 4)(x^4 - 1)$.

Now I need to factor each of these factors further, if possible, over the integers.

For the first factor, $x^4 + 4$, I recall the Sophie Germain identity, which says that $a^4 + 4b^4 = (a^2 + 2b^2 + 2ab)(a^2 + 2b^2 - 2ab)$.

Applying this identity with $a = x$ and $b = 1$, I get $x^4 + 4 = (x^2 + 2x + 2)(x^2 - 2x + 2)$.

For the second factor, $x^4 - 1$, I can use the difference of squares formula, which says that $a^2 - b^2 = (a + b)(a - b)$.

Applying this formula with $a = x^2$ and $b = 1$, I get $x^4 - 1 = (x^2 + 1)(x^2 - 1)$.

Then I can apply the difference of squares formula again with $a = x$ and $b = 1$, to get $x^2 - 1 = (x + 1)(x - 1)$.

So the original polynomial is $(x^4 + 4)(x^4 - 1) = (x^2 + 2x + 2)(x^2 - 2x + 2)(x^2 + 1)(x + 1)(x - 1)$.

These are all the irreducible factors over the integers, so $p_1(x) = x^2 + 2x + 2$, $p_2(x) = x^2 - 2x + 2$, $p_3(x) = x^2 + 1$, $p_4(x) = x + 1$, and $p_5(x) = x - 1$.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) = 1^2 + 2 \cdot 1 + 2 = 5$, $p_2(1) = 1^2 - 2 \cdot 1 + 2 = 1$, $p_3(1) = 1^2 + 1 = 2$, $p_4(1) = 1 + 1 = 2$, and $p_5(1) = 1 - 1 = 0$.

So the final answer is $5 + 1 + 2 + 2 + 0 = 10$.

Answer: 10

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x)\cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

This problem is about factoring a polynomial over the integers, which can be a tricky task.

One possible strategy is to look for rational roots of the polynomial, using the rational root theorem.

However, that would involve checking a lot of possible factors of -4, and there is no guarantee that the polynomial has any rational roots at all.

Another possible strategy is to use some algebraic tricks to simplify the polynomial or reduce its degree.

I notice that the polynomial has only even powers of x, so I can try to substitute $y = x^2$ and see what happens.

Then the polynomial becomes $y^4 + 3y^2 - 4$, which is a quadratic equation in y^2.

I can try to factor this quadratic equation by looking for two numbers that multiply to -4 and add to 3.

Those numbers are 4 and -1, so I can write $y^4 + 3y^2 - 4 = (y^2 + 4)(y^2 - 1)$.

Now I can substitute back $x^2 = y$ and get $x^8 + 3x^4 - 4 = (x^4 + 4)(x^4 - 1)$.

This looks promising, because I have reduced the degree of the factors from 8 to 4.

I can try to factor each of these factors further, using the difference of squares formula.

I get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x^2 - 1)$.

I can apply the difference of squares formula again to the last factor and get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x + 1)(x - 1)$.

Now I have factored the polynomial completely into monic linear and quadratic factors with integer coefficients.

These are the $p_i(x)$'s that the problem is asking for.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (1^2 + 2)(1^2 - 2)(1^2 + 1)(1 + 1)(1 - 1)$.

Simplifying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (3)(-1)(2)(2)(0)$.

Multiplying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = 0$.

Answer: 0

Figure 2: Two solutions to the same problem, graded by the PRM. The solution on the left is correct while the solution on the right is incorrect. A green background indicates a high PRM score, and a red background indicates a low score. The PRM correctly identifies the mistake in the incorrect solution.

log-likelihood of these target tokens during training. The PRM can therefore be trained in a standard language model pipeline without any special accommodations. To determine the step-level predictions at test time, it suffices to perform a single PRM forward pass over the whole solution. We visualize large-scale PRM scores for two different solutions in Figure 2. To compare multiple solutions, it is necessary to compute a single score for each solution. This is an important but straightforward detail: we define the PRM score for a solution to be the probability that every step is correct under the PRM. We implement this as the product of the correctness probabilities for each step. We describe other possible scoring strategies and additional PRM training details in Appendix F.

When we provide process supervision, we deliberately choose to supervise only up to the first incorrect step. This makes the comparison between outcome and process supervision more straightforward. For correct solutions, both methods provide the same information, namely that every step is correct. For incorrect solutions, both methods reveal the existence of at least one mistake, and process supervision additionally reveals the precise location of that mistake. If we were to provide additional process supervision beyond the first mistake, then process supervision would have an even greater information advantage. This decision also keeps the labelling cost similar for humans: without relying on an easy-to-check final answer, determining the correctness of a solution is equivalent to identifying its first mistake. While most MATH problems do have easy-to-check final answers, we expect this to not remain true in more complex domains.